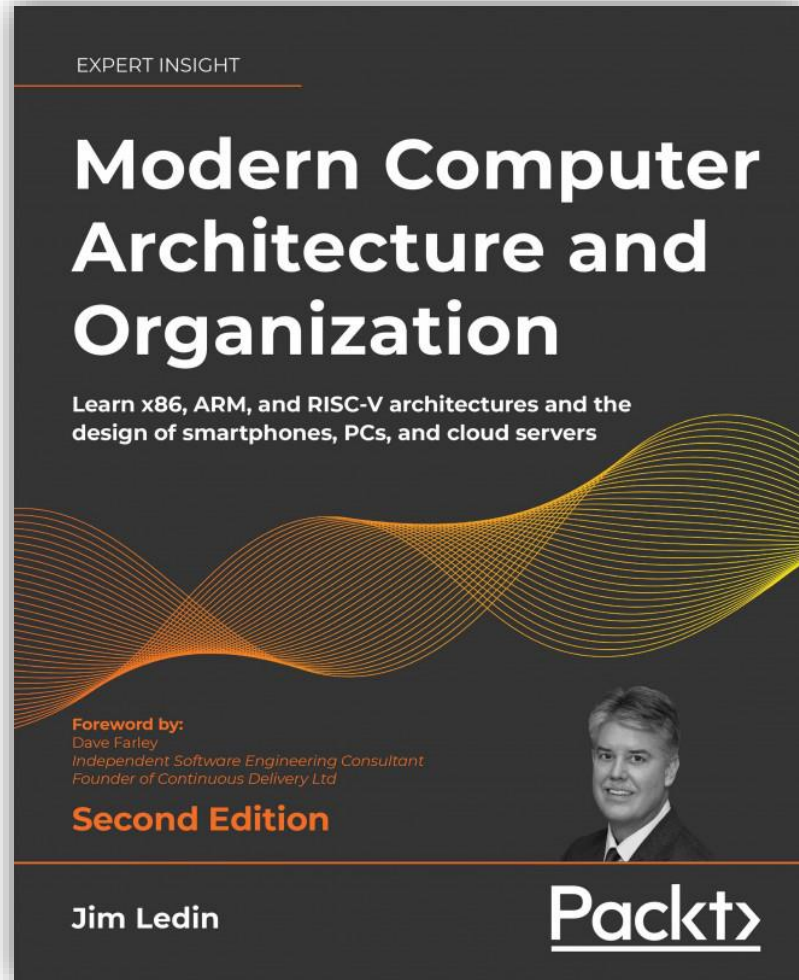


Computer Architecture

Chapter 3 – Processor Elements



VIVES University of Applied Sciences Bachelor in electronics-ICT



— Computer Architecture

Chapter 3: Processor Elements

Contents

1. **A simple processor**
2. Addressing modes
3. Instruction categories
4. Interrupt processing
5. Input/output operations

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor

- The **6502** processor architecture and a small subset of its instructions were introduced in *Chapter 1, Introducing Computer Architecture*.
- In this section, we will build upon that introduction to present the generic functional components universally employed in processor architectures, from the tiniest embedded controllers to the most powerful server CPUs.
- The integrated circuit at the core of a computer system goes by a few different names: the **Central Processing Unit** (CPU), **microprocessor**, or, simply, **processor**.
- A microprocessor is a single integrated circuit that implements the functions of a processor.
- A typical processor contains three logically distinct functional units:
 - The **control unit**, which manages the overall operation of the device: This management activity includes **fetching** the next instruction from memory, **decoding** the instruction to determine the operation to perform, and distributing the **execution** of the instruction to appropriate elements within the processor.
 - The **Arithmetic Logic Unit** (ALU): This is a combinational circuit that executes **arithmetic** and **bit manipulation operations**.
 - The **register set**: This provides temporary storage as well as **source** and **destination** locations for instruction inputs and outputs.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor

- A typical processor contains three logically distinct functional units:
 - The **control unit**, which manages the overall operation of the device: This management activity includes **fetching** the next instruction from memory, **decoding** the instruction to determine the operation to perform, and distributing the **execution** of the instruction to appropriate elements within the processor.
 - The **Arithmetic Logic Unit** (ALU): This is a combinational circuit that executes **arithmetic** and **bit manipulation operations**.
 - The **register set**: This provides temporary storage as well as **source** and **destination** locations for instruction inputs and outputs.
- The following diagram shows the flow of control and data among the control unit, the registers, the ALU, system memory, and input/output devices (**Figure 3.1**):

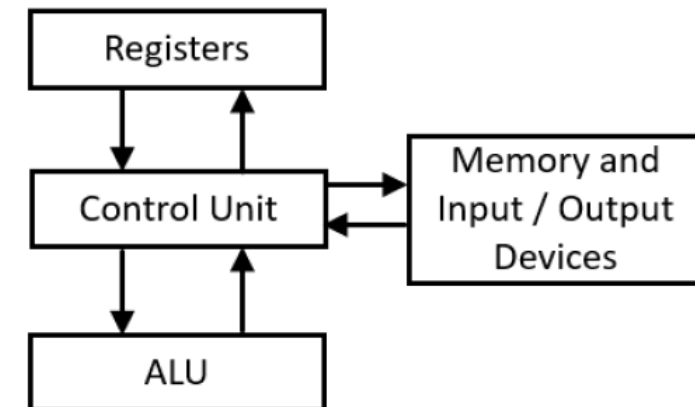


Figure 3.1: flow of control and data

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Control Unit

- The **control unit** of a modern processor is a **synchronous sequential** digital circuit.
- Its function is to interpret processor instructions and manage the execution of those instructions by interacting with the other functional units within the processor and with external components, including memory and input/output devices.
- The control unit is a key part of the **von Neumann architecture**.
- When a computer system is **powered on**, the processor undergoes a **reset process** to initialize its internal components to defined values. During the reset process, it loads the **Program Counter (PC)** register with the memory location of the first instruction to be executed.
- Software developers who construct the lowest-level system software components must configure their development tools to produce a code memory image that begins execution at the address required by the processor architecture.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Control Unit

- The **Program Counter** (PC) is a central component of the **control unit**. The PC register contains the memory address of the **next** instruction to be executed.
- At the beginning of the instruction execution cycle, the control unit reads the data word at the memory address indicated by the PC and loads it into an internal register for **decoding** and **execution**.
- The first word of an instruction contains an **opcode**. Based on the opcode bit pattern, the control unit may read additional memory locations following the opcode to retrieve data needed by the instruction, such as a **memory address** or **data operand**.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Control Unit

- As the **control unit** completes the **reset process** and begins executing instructions, it performs the repetitive cycle shown in **Figure 3.2**.
- Once the **reset** has completed, the PC register contains the initial instruction location.
- The control unit fetches the **first instruction** from memory and **decodes** it. Decoding is the process the control unit performs to determine the actions required by the instruction. As part of the decoding process, the control unit identifies the **category of instruction**.

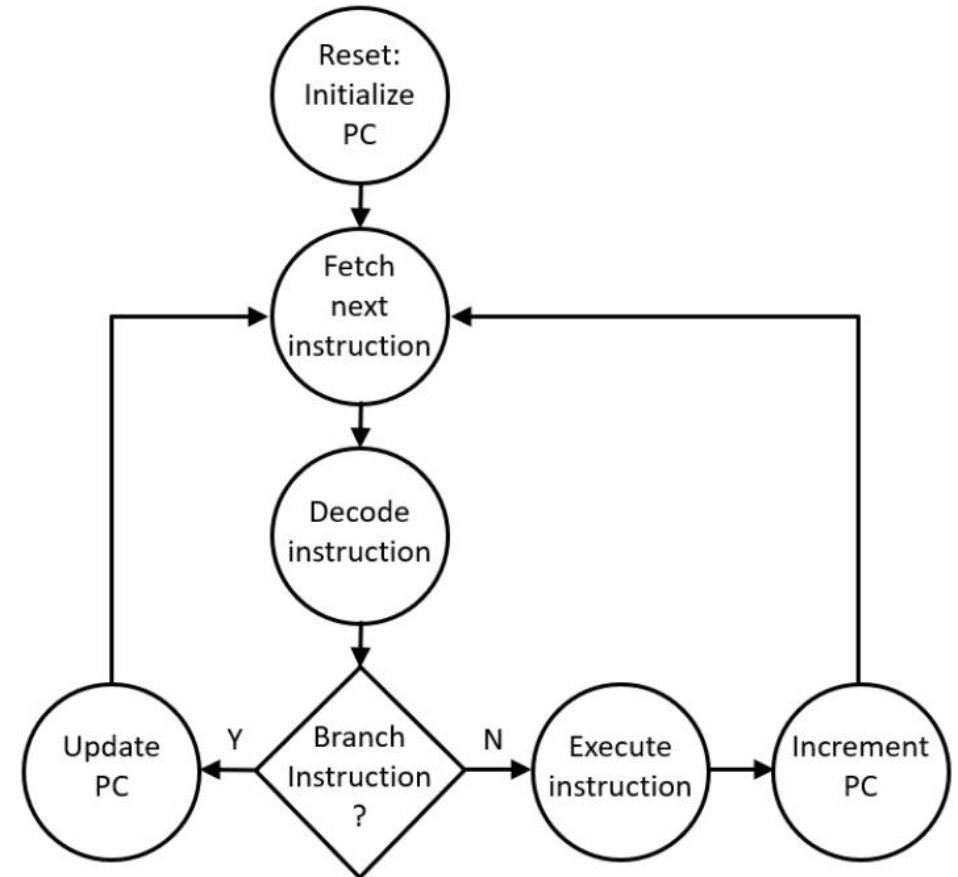


Figure 3.2: instruction execution cycle

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Control Unit

- The two basic instruction categories represented in **Figure 3.2** are **branching instructions** and **all other instructions**.
- **Branching instructions** are implemented directly by the control unit. These instructions cause the contents of the PC to be replaced with the memory address of the branch destination.
- Examples of instructions that perform branching are **conditional branch instructions** (when the branch is taken), **subroutine calls**, **subroutine returns**, and **unconditional branching** (also called *jump*) instructions.

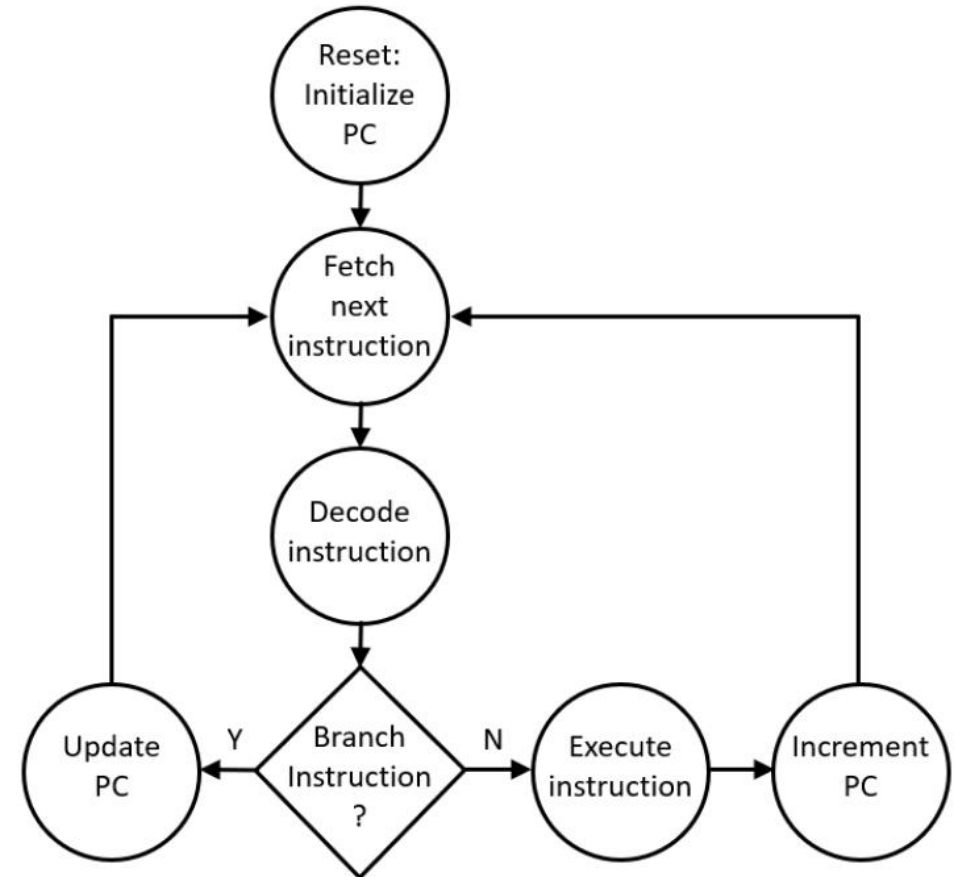


Figure 3.2: instruction execution cycle

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Control Unit

- Instructions that do *not* involve branching are carried out by processor circuitry under the direction of the **control unit**.
- The steps in executing an instruction include tasks such as reading or writing a register, reading or writing a memory location, directing the ALU to perform a mathematical operation, or other miscellaneous activities.
- In most processors, the execution of a single instruction takes place over **multiple processor clock cycles**.
- The instruction clock cycle count can vary significantly from simple instructions that require only a small number of clock cycles to complex operations that take many cycles to complete.
- The control unit orchestrates all of this activity.

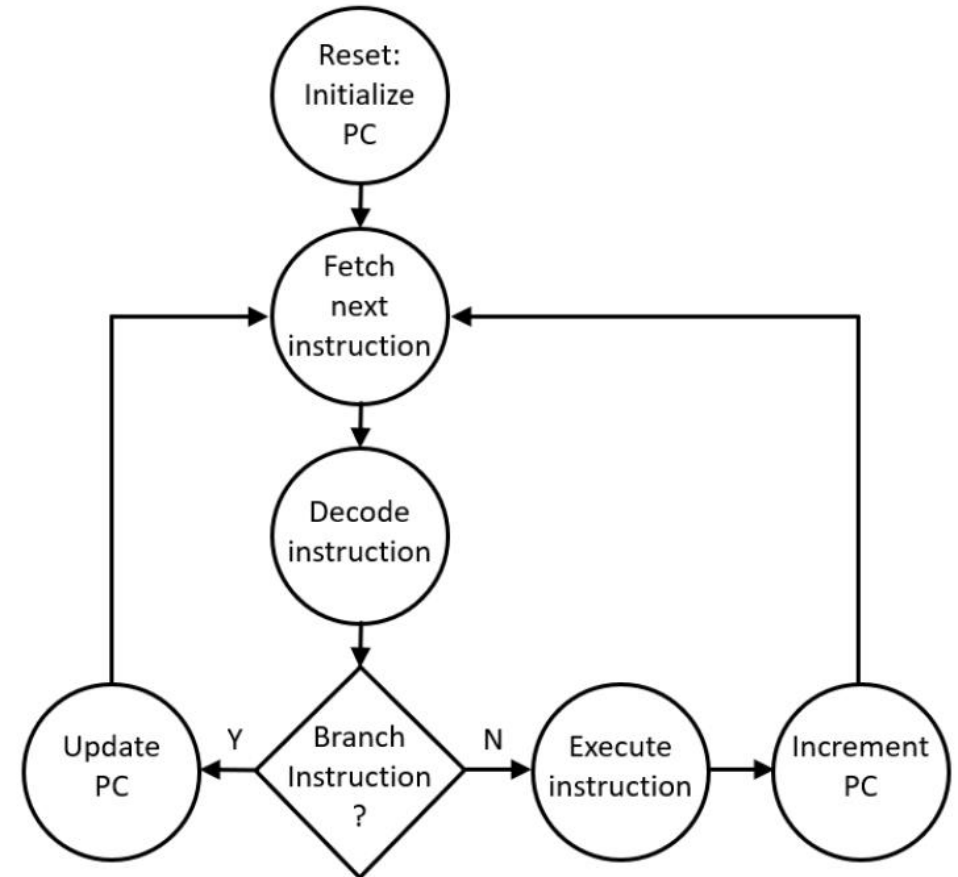


Figure 3.2: instruction execution cycle

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Executing an instruction

- Consider a simplified example of two 6502 instructions, **TXA** and **TYA**.
- **TXA** copies the contents of register **X** to register **A**, and **TYA** does the same thing using the **Y** register as the source.
- If we consider these two instructions in isolation, the execution of both instructions can be implemented as shown in **Figure 3.3**.

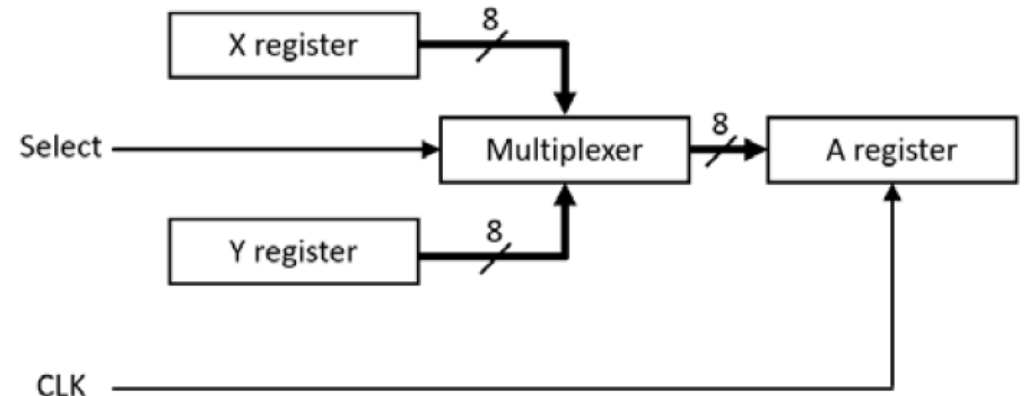


Figure 3.3: 6502 TXA & TYA instructions

- To execute the **TXA** instruction, these steps are performed:
 1. The control unit first sets the Select input to direct the **X** register data bits through to the output of the multiplexer. This presents the data from X at the inputs to the **A** register.
 2. After the Select input to the multiplexer has been set, the control unit must pause for the propagation of the data bits to the multiplexer outputs.
 3. After the multiplexer outputs have stabilized, the control unit generates a rising edge on the CLK signal to load the X register data bits into register **A**.
- To execute the **TYA** instruction, the control unit performs the same sequence of steps, except it initially sets the Select input to feed the Y register to the multiplexer output.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- The ALU performs **arithmetic** and **bit-oriented operations** under the direction of the **control unit**.
- To perform an operation, the ALU requires data input values, called **operands**, and a **code** indicating the operation to be performed. The ALU output is the result of the operation.
- ALU operations may use one or more **processor flags**, such as the carry flag, as input and set the states of processor flags as outputs.
- In the 6502, ALU operations update the **carry**, **negative**, **zero**, and **overflow** flags.
- An ALU is a **combinational** circuit, implying the outputs update **asynchronously** in response to changes at the inputs and that it retains no memory of previous operations.
- To execute an instruction involving the ALU, the control unit applies inputs to the ALU, pauses for the propagation delay across the ALU, then transfers the ALU output to the destination specified by the instruction.
- The ALU contains an **adder** circuit to perform **addition** and **subtraction** operations.
- In a processor with **two's complement arithmetic**, **subtraction** can be implemented by first performing a two's complement negation of the right operand and adding that result to the left operand.
- Mathematically, when performing subtraction in this manner, the expression $A - B$ is transformed into $A + (-B)$.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- The **two's complement negation** of a signed number is achieved by **inverting** all of the bits in the operand and **adding 1** to the result. Incorporating this operation, subtraction represented as $A+(-B)$ becomes $A+(\text{NOT}(B)+1)$.
- Looking at subtraction in this form should clarify the use of the 6502 carry flag in conjunction with the **SBC** instruction clear.
- The **C flag** provides the "+1" in subtraction when there is *no borrow*. If there is a **borrow**, the sum must be reduced by 1, which is accomplished by setting the **C** flag to **0**.
- *To summarize, in the 6502, subtraction logic is identical to addition logic with the single difference that the B operand in A-B is routed through a set of NOT gates to invert all eight of the bits prior to feeding NOT(B) to the adder input.*

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- Figure 3.4 is a functional representation of the **addition** and **subtraction** operations in the 6502:

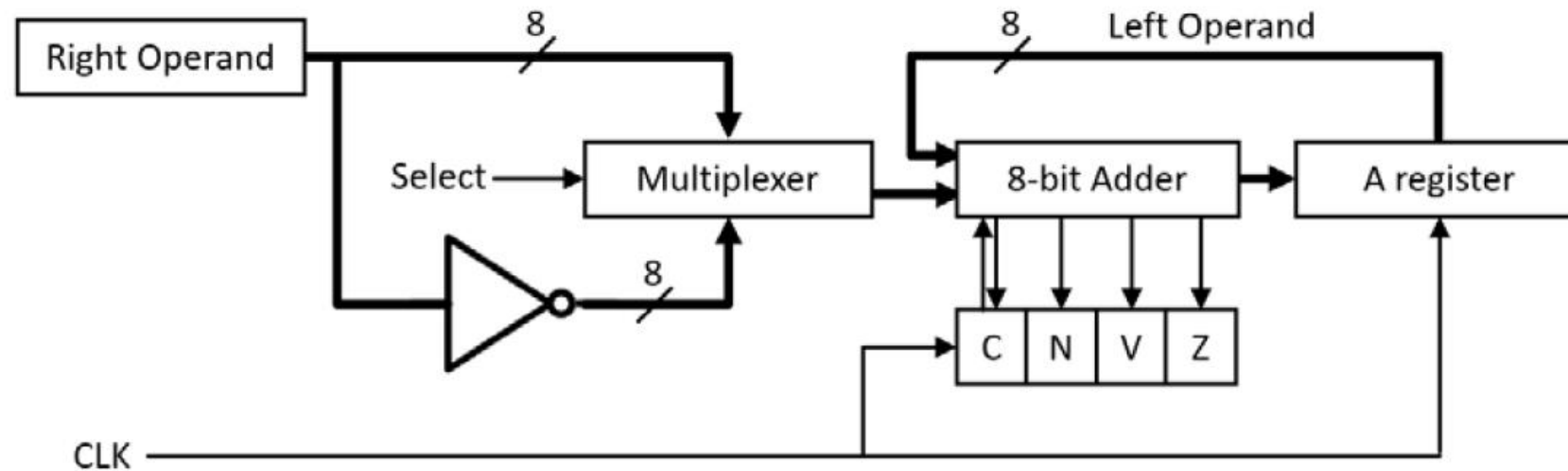


Figure 3.4: 6502 addition and subtraction operations

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- **Figure 3.4** is a highly simplified representation of the 6502 processor depicting only the components involved in the **ADC** and **SBC** instructions.
- The **Select** input in **Figure 3.4** chooses whether the operation is **addition** or **subtraction**.
- **Addition** requires selecting the upper multiplexer input, while the lower multiplexer input selects **subtraction**.
- In the 6502 architecture, the **A** register is **always** the **left** operand for **subtraction**!

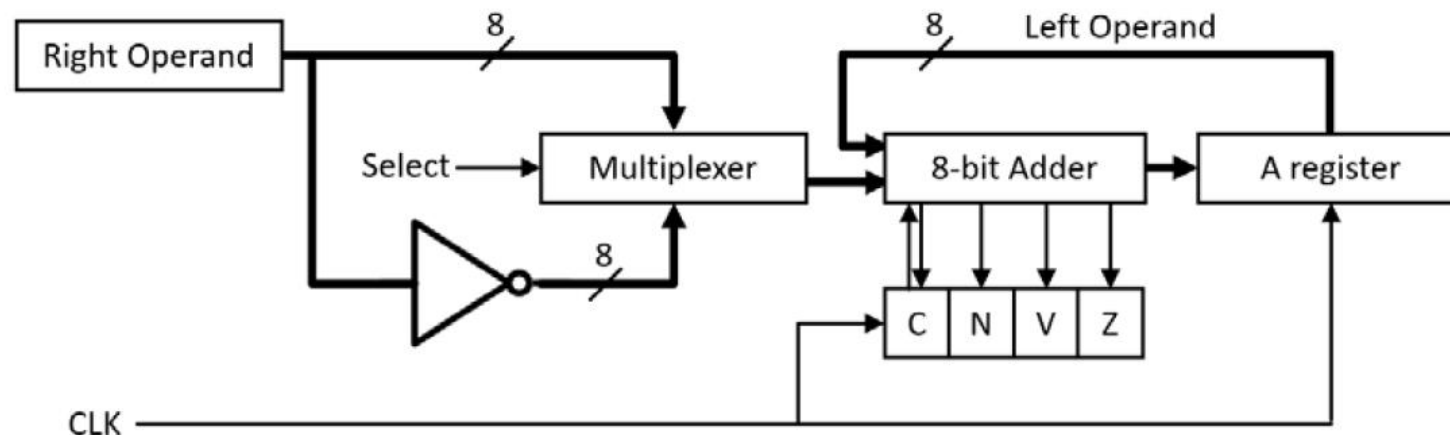


Figure 3.4: 6502 addition and subtraction operations

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- The inputs to the adder are the **left** and **right operands** and the **C flag**. When executing an **ADC** or **SBC** instruction, the control unit presents the **right** operand to the multiplexer data inputs and sets the multiplexer select input to the appropriate state for the given instruction.
- After pausing for propagation through the NOT gate, the multiplexer, and the adder, the control unit generates a **clock edge** that latches the adder outputs into the A register and the processor flags register.

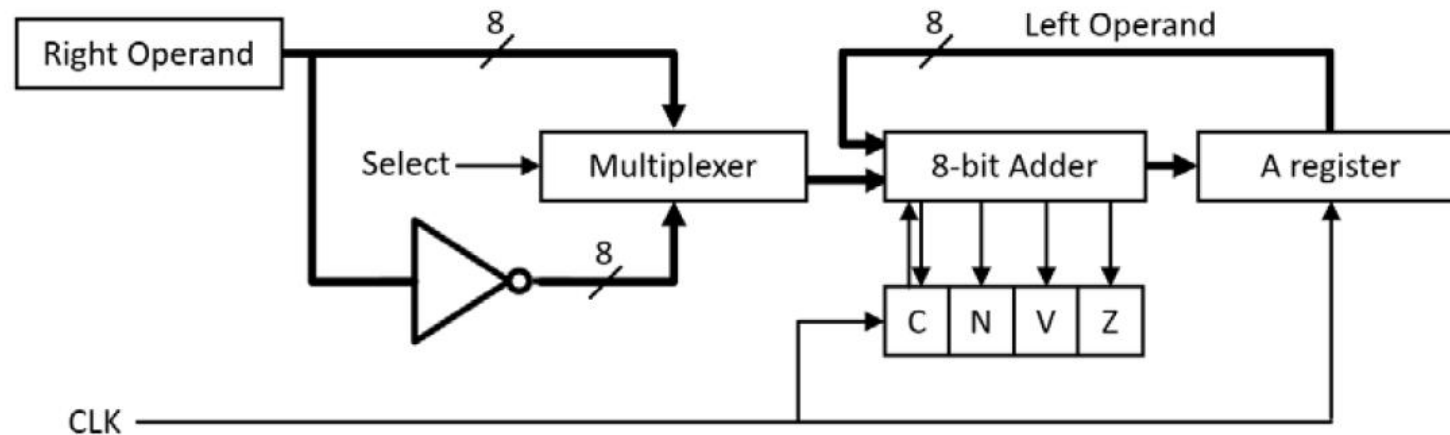


Figure 3.4: 6502 addition and subtraction operations

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- The processor flags are set as shown in the following execution of an **ADC** or **SBC** instruction:
 - C** indicates whether an **addition** generated in a **carry** ($C = 1$) or whether a **subtraction** produced a **borrow** ($C = 0$).
 - N** contains the value of bit 7 of the result.
 - V** indicates whether a **signed overflow** occurred ($V = 1$ if overflow occurred).
 - Z** is 1 if the result is **zero**. Z is 0 otherwise.

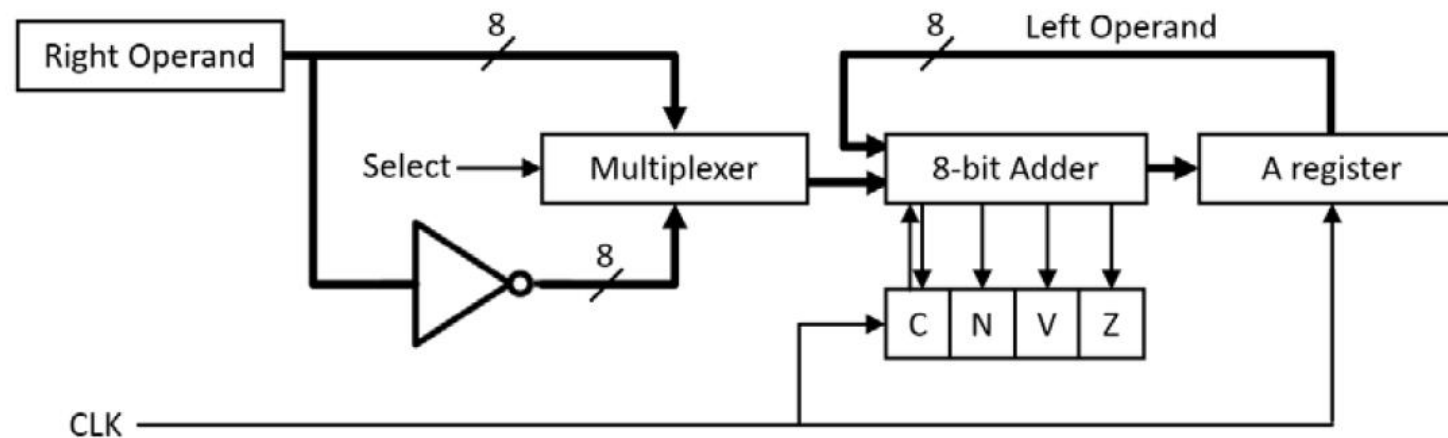


Figure 3.4: 6502 addition and subtraction operations

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- An ALU supports a variety of operations in addition to adding and subtracting two numbers.
- *In the 6502, operations with two operands generally use the **A** register as the **left** operand.*
- The **right** operand is either read from a **memory location** or provided as an **immediate value** in the next memory location after the opcode.
- All 6502 ALU operands and results are **8-bit** values.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Arithmetic Logic Unit

- The 6502 ALU operations are as follows:
 - **ADC, SBC**: Add or subtract two operands with **carry** input.
 - **DEC, DEX, DEY**: Decrement a memory location or register by one.
 - **INC, INX, INY**: Increment a memory location or register by one.
 - **AND**: Perform a bitwise logical AND operation on two operands
 - **ORA**: Perform a bitwise logical OR operation on two operands.
 - **EOR**: Perform a bitwise logical XOR operation on two operands.
 - **ASL, LSR**: Shift the A register or memory location left or right by one bit position and insert 0 into the vacated bit position.
 - **ROL, ROR**: Rotate the A register or memory location left or right by one bit position and insert the **C** flag value into the vacated bit position.
 - **CMP, CPX, CPY**: Subtract two operands and *discard the result*, setting the **N**, **Z**, and **C** flags based on the result of the subtraction.
 - **BIT**: Perform a bitwise logical AND between two operands and use the **Z** flag to indicate whether the result is 0. In addition, copy bits 7 and 6 of the left operand to the **N** and **V** flags.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: Registers

- **Processor registers** are internal storage locations that serve as **sources** and **destinations** for instruction operations. Registers provide the quickest data access in a processor, but are limited to a very small number of locations.
- The width of a register in bits is generally the same as the processor word size.
- The 6502, as we have seen, has only **three 8-bit registers**: **A**, **X**, and **Y**.
- When designing a new processor architecture, it is critical to evaluate the tradeoff between the **number** of **registers** and the **number** and **complexity** of **instructions** available to the processor.
- For a given integrated circuit die size and fabrication process (which together define the number of transistors available for the processor), adding more **registers** to the architecture reduces the number of transistors available for executing instructions and other functions.
- In contrast, adding instructions with **complex** capabilities may limit the die space available for registers.
- This tension between *instruction set complexity* and the *number of registers* is expressed in the categorization of an architecture as **CISC** or **RISC**.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: CISC & RISC

- **CISC** (*Complex Instruction Set Computer*) processors are characterized as having a **rich instruction set** providing a variety of features, such as the ability to load operands from memory, perform an operation, and store the result to memory, all in one instruction.
- In a CISC processor, an instruction may take **many clock cycles** to execute as the processor performs all of the required subtasks.
- CISC processors tend to have a **smaller number of registers** due, in part, to the die space required for the circuitry occupied by the instruction set logic.
- The **x86** is a classic example of CISC architecture.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: CISC & RISC

- **RISC** (*Reduced Instruction Set Computer*) processors, on the other hand, have a **smaller number of instructions** that each perform simpler tasks in comparison to CISC instructions.
- Performing an operation on data values stored in memory might require a pair of load instructions to load two operands from memory into registers, another instruction to perform the operation, and a final instruction to store the result back to memory.
- The **key difference between CISC and RISC** is that **RISC architectures** are optimized *to execute individual instructions at very high speed*. Even though reading memory, performing the operation, and writing the result back to memory require several more instructions in a RISC processor than in a CISC processor, the overall start-to-finish time may be comparable or even faster for the RISC processor.

— Computer Architecture

Chapter 3: Processor Elements

1. A simple processor: CISC & RISC

- Examples of **RISC** architectures are **ARM**, discussed in *Chapter 10, Modern Processor Architectures and Instruction Sets*, and **RISC-V**, discussed in *Chapter 11, The RISC-V Architecture and Instruction Set*.
- The reduction in instruction set complexity in RISC processors leaves more die space available for **registers**, which generally results in a larger number of registers in RISC processors compared to CISC.
- The **ARM** architecture has **13** general-purpose registers, while the **RISC-V** base 32-bit architecture has **31** general-purpose registers.
- The **larger number of registers** in **RISC architectures** reduces the need to access system memory because more registers are available for storage of intermediate results. This helps with performance because accessing system memory is significantly *more time consuming* than accessing data located in processor registers.

— Computer Architecture

Chapter 3: Processor Elements

Contents

1. A simple processor
- 2. Addressing modes**
3. Instruction categories
4. Interrupt processing
5. Input/output operations

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **CISC** processors support **multiple addressing modes** for instructions that require transferring data between memory and registers. **RISC** processors have a **more limited number of addressing modes**.
- Each processor architecture defines its collection of addressing modes based on an analysis of the anticipated memory access patterns that software will use on that architecture.

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Immediate addressing mode**
- For the first example, assume we are given the values of the **four bytes to sum** and asked to write a 6502 program to perform that task. This allows us to enter the byte values directly into our code. The bytes in this example are **\$01** through **\$04**.
- We'll be adding the bytes together in **reverse** order (\$04 down to \$01) in anticipation of the use of a looping construct later in this section.
- This code uses **immediate addressing mode** to add the four bytes together:

```
; Add four bytes together using immediate addressing mode
LDA #$04
CLC
ADC #$03
ADC #$02
ADC #$01
```

Figure 3.5: Example of Immediate addressing mode

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Absolute addressing mode**
- **Absolute addressing mode** specifies the **memory location** containing the value to be read or written by the instruction.
- The 6502 has **16 address bits**, so an address field that supports accessing all available memory is **two bytes long**.
- A complete instruction to access data at an arbitrary 6502 memory location consists of **three bytes**: the **first** byte is the **opcode** followed by **two bytes** for the address to be read or written.
- The two address bytes must be stored with the **lower-order byte first**, followed by the **higher-order byte**.
- The convention of storing the lower-order byte of a two-byte address at the lower memory address makes the 6502 a **little-endian** processor. The **x86** and **RISC-V** are also **little-endian**.
- The **ARM** architecture allows selection of **big- or little-endian mode** under software control (this is called bi-endianness), but *most operating systems* running on the **ARM architecture** select **little-endian mode**.

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Absolute addressing mode**
- For the absolute addressing mode example, we begin with some setup code to store the four bytes to be added together into addresses **\$200** through **\$203**.
- The instructions to add the four bytes follow the setup code.
- This example uses **absolute addressing mode** to sum the four bytes:

```
; Initialize the data in memory
LDA #$04
STA $0203
LDA #$03
STA $0202
LDA #$02
STA $0201
LDA #$01
STA $0200

; Add four bytes together using absolute addressing mode
LDA $0203
CLC
ADC $0202
ADC $0201
ADC $0200
```

Figure 3.6: Example of absolute addressing mode

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Absolute indexed addressing mode**
- **Absolute indexed addressing** mode computes a memory address by **adding** a **base address** provided in the instruction to a value contained in the **X** or **Y** register.
- The following example adds the bytes at addresses \$0200 through \$0203 using absolute indexed addressing. The **X register** provides an **offset** from the base of the byte array at address \$0200:

```
; Initialize the data in memory
LDA #$04
STA $0203
LDA #$03
STA $0202
LDA #$02
STA $0201
LDA #$01
STA $0200
```

```
; Add four bytes together using absolute indexed addressing
mode
LDX #$03
LDA $0200, X
DEX
CLC
ADD_LOOP:
ADC $0200, X
DEX
BPL ADD_LOOP
```

Figure 3.7: Example of absolute indexed addressing mode

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Absolute indexed addressing mode**

```
; Initialize the data in memory
LDA #$04
STA $0203
LDA #$03
STA $0202
LDA #$02
STA $0201
LDA #$01
STA $0200
```

```
; Add four bytes together using absolute indexed addressing
mode
LDX #$03
LDA $0200, X
DEX
CLC
ADD_LOOP:
ADC $0200, X
DEX
BPL ADD_LOOP
```

Figure 3.7: Example of absolute indexed addressing mode

- The **DEX** instruction decrements (subtracts 1 from) the X register.
- The **BPL** instruction means "branch on plus", which conditionally transfers control to the instruction preceded by the ADD_LOOP label. **BPL** executes the branch only when the processor **N** flag is **clear**. If the N flag is **set**, BPL continues with the next instruction in memory.

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Indirect indexed addressing mode**
- **Indirect indexed addressing** mode uses a two-byte address stored in the memory address range \$00-\$FF as the **base address** and **adds** the contents of the **Y** register to that base to produce the memory address used by the instruction.
- In the following example, the **base address** of the byte array (\$0200) is first stored in **little-endian** order at addresses \$0010 and \$0011.
- The code uses **indirect indexed addressing** in a loop to add the bytes together:

```
; Initialize the data in memory
LDA #$04
STA $0203
LDA #$03
STA $0202
LDA #$02
STA $0201
LDA #$01
STA $0200
```

```
; Initialize the pointer to the byte array
LDA #$00
STA $10
LDA #$02
STA $11
```

```
LDY #$03
LDA ($10), Y
DEY
CLC
ADD_LOOP:
ADC ($10), Y
DEY
BPL ADD_LOOP
```

Figure 3.8: Example of indirect indexed addressing mode

— Computer Architecture

Chapter 3: Processor Elements

2. Addressing modes

- **Zero-page addressing mode**
- The 6502 has some other addressing modes available: **zero-page addressing mode** provides instructions that are smaller (one byte less in length) and **faster** to execute for absolute and absolute indexed addressing by working only with memory addresses in the range **\$00-\$FF**.
- The term zero-page refers to the **high byte of the 16-bit address**, which is **zero** for addresses in this range. Other than exhibiting improved performance in terms of faster execution speed and reduced code memory usage, the zero-page addressing modes behave the same as the corresponding addressing modes described earlier.
- **Indexed indirect addressing mode**
- Another mode is called **indexed indirect addressing mode**, which is similar to **indirect indexed** addressing, except the **X register is used instead of Y** and the **offset** contained in **X** is **added** to the **address** provided in the instruction to determine the address of the pointer to the data.
- For example, assume X contains the value 8.
- The LDA (\$10, X) instruction will add the contents of X to \$10, producing the result \$18.
- The instruction then uses the 16-bit memory address read from addresses **\$18-\$19** as the target memory address to load into the A register.

— Computer Architecture

Chapter 3: Processor Elements

Contents

1. A simple processor
2. Addressing modes
- 3. Instruction categories**
4. Interrupt processing
5. Input/output operations

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Memory load and store instructions**
- The 6502 uses load and store instructions to read data values from system memory into processor registers and to write registers out to system memory.
- In the 6502 architecture, the **LDA**, **LDX**, and **LDY** instructions **load** the register identified in the instruction with an **8-bit** word from system memory.
- **LDA** supports **all addressing modes** available in the 6502, while **LDX** and **LDY** each support a **more limited subset of addressing modes**: immediate, absolute, and absolute indexed.
- After each of these instructions finishes executing, the **N** and **Z** flags indicate whether the value that was loaded is **negative** (that is, bit 7 equals 1) and whether the value is **zero**.
- **STA**, **STX**, and **STY** **store** the register identified in the instruction to memory.
- Each store instruction supports the same addressing modes as the load instruction for that register, except the store instructions **do not support immediate addressing mode**.
- These instructions update the **N** and **Z** flags to reflect the value stored.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Register-to-register data transfer instruction**
- These instructions copy an **8-bit** word from one of the **A**, **X**, and **Y** registers to another register.
- These instructions use **implied addressing mode**, which means the source and destination of each instruction are indicated directly by the **instruction opcode**.
- **TAX** copies the A register contents to the X register.
- **TAY**, **TXA**, and **TYA** perform similar operations between the register pairs indicated by the instruction mnemonic.
- These instructions update the **N** and **Z** flags.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Stack instructions**
- The **TXS** instruction copies the X register to the **stack pointer** (S) register. This instruction must be used to initialize the S register during **system startup**.
- **TSX** copies the S register to the X register. **TSX** updates the **N** and **Z** flags. **TXS** does *not affect* any flags.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Stack instructions**
- **PHA** pushes the **A register** contents onto the stack. **PHP** pushes the **processor flags** onto the stack as an 8-bit word. *These instructions do not affect the processor flags.* Pushing a value onto the stack consists of writing the register to the memory address computed by **adding \$100 to the S register**, then decrementing the S register.
- **PLA** and **PLP** **pop** the A register and the flags register from the stack respectively. Popping a value first increments the S register then transfers the value at the location computed by **adding \$100 to the S register** to the target register location.
- **PLA** updates the **N** and **Z** flags. **PLP** *sets or clears six of the seven processor flags* based on the value popped from the stack.
- The **B** (break) flag is only meaningful in a copy of the processor flags register that has been pushed onto the stack by an **interrupt** or by the **PHP instruction**. This distinguishes a **BRK** instruction from a *hardware* interrupt request.
- Both the **PHP** and **BRK** instructions push the flags register with the **B bit** (bit 4) **set**. Hardware interrupts generated via the processor **IRQ (Interrupt Request)** and **NMI (Non-Maskable Interrupt)** pins push the processor flags register with the **B bit cleared**.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Arithmetic instructions**
- Addition and subtraction are performed by the **ADC** and **SBC** instructions. The **left** operand to each instruction is the **A register**, which is also the **destination** for the result of the operation.
- All addressing modes are available for designating the right operand. The **Z**, **C**, **N**, and **V** flags are updated to reflect the result of the operation.
- **INC** and **DEC**, respectively, increment or decrement the specified memory location by adding 1 to, or subtracting 1 from, the value at that location. The result is stored at the same memory location. Absolute and absolute indexed addressing modes are supported. These instructions update the **N** and **Z** flags based on the result of the operation.
- The **INX**, **DEX**, **INY**, and **DEY** instructions increment or decrement the **X** or **Y** register, as indicated by the mnemonic. These instructions update the **N** and **Z** flags.
- The **CMP** instruction performs a comparison by subtracting the operand value from the **A** register. The behavior of **CMP** is very similar to the instruction sequence **SEC** followed by **SBC**. The **N**, **Z**, and **C** flags are set to reflect the result of the subtraction.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Arithmetic instructions**
- The differences between **CMP** and **SBC** are as follows:
 - CMP discards the result of the subtraction (the value in A is unaffected by the CMP instruction).
 - CMP does not use decimal mode if the D flag is set.
 - CMP does not affect the value of the V flag.
 - CMP supports all addressing modes.
- The **CPX** and **CPY** instructions are similar to **CMP** except the X or Y register is used as the **left** operand as indicated in the mnemonic and only absolute and absolute indexed addressing modes are supported for the right operand.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Logical instructions**
- The **AND**, **EOR**, and **ORA** instructions perform bitwise AND, XOR, and OR operations respectively between the A register and the operand. The result is stored in the A register.
- The **Z** and **N** flags are updated to reflect the result of the operation. All addressing modes are supported.
- The **ASL** instruction shifts the operand one bit left, inserting a **zero** as the **least significant bit**. The **most significant bit** is shifted into the **C** flag. This is equivalent to **multiplying** the A register **by two** and placing the most significant bit of the 9-bit result in C.
- Similar to ASL, **LSR** shifts the operand one bit right, inserting a **zero** as the **most significant bit**. The least significant bit is shifted into the **C** flag. This is equivalent to **division by two** of an unsigned operand, with the remainder placed in C.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Logical instructions**
- The **ROL** and **ROR** instructions shift the A register one bit to the left or right, respectively. The previous value of the **C** flag is shifted into the bit location vacated by the shift operation. The bit shifted out of A is stored in the **C** flag.
- **ASL**, **LSR**, **ROL**, and **ROR** support accumulator addressing mode, which uses the A register as the operand. This mode is specified by using the special operand value "A", as in **ASL A**. These four instructions also support absolute and absolute indexed addressing modes.
- The **BIT** instruction performs a **bitwise AND** between the operand and the A register, and *the result is discarded*. The **Z** flag is updated based on the result of this operation. Bits 7 and 6 from the memory location are copied to the **N** and **V** flags, respectively. Only absolute addressing mode is supported.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Branching instructions**
- The **JMP** instruction loads the operand into the PC register and continues execution from the instruction at that location. The destination is a two-byte absolute address and can be anywhere in the 6502's 16-bit address space.
- The **BCC** and **BCS** instructions perform conditional branching if the **C** flag is clear or set, respectively.
- The **BNE** and **BEQ** instructions perform conditional branching if the **Z** flag is clear or set, respectively.
- The **BPL** and **BMI** instructions perform conditional branching if the **N** flag is clear or set, respectively.
- The **BVC** and **BVS** instructions perform conditional branching if the **V** flag is clear or set, respectively.
- The conditional branch instructions use **relative addressing mode**, which is a **signed 8-bit offset** (in the range -128 to +127 bytes) from the address of the instruction following the branch instruction.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Subroutine call and return instructions**
- The **JSR** instruction pushes the address of the instruction following the JSR instruction (minus one) onto the stack, then loads the address specified as the 16-bit operand into the PC and continues execution from the instruction at that location.
- **RTS** is used to end a subroutine.
- The return PC value (minus one) is pulled from the stack and loaded into the PC register.
- The RTS instruction increments the PC register after pulling it from the stack, before it is used as the address of the next instruction to execute.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Processor flag instructions**
- The **SEC** and **CLC** instructions set and clear the **C** flag, respectively.
- The **SED** and **CLD** instructions set and clear the **D** flag, respectively.
- The **CLV** instruction clears the **V** flag.
- *There is no instruction that simply sets the V flag!*

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **Interrupt related instructions**
- The **SEI** and **CLI** instructions set and clear the **I** flag, respectively. When the **I** flag is **set**, **maskable interrupts** are **disabled**.
- The **BRK** instruction triggers a **non-maskable interrupt**.
- The memory address two bytes after the **BRK** instruction is pushed onto the stack, followed by the processor flags register. The PC register is loaded with the interrupt service routine address, which is read from memory addresses **\$FFFE-\$FFFF**. **The interrupt service routine** then begins to execute.
- The **BRK** instruction does **not** alter any register contents (other than the stack pointer) or processor flags. The flags register pushed onto the stack has the **B** bit **set** to indicate the interrupt is the result of a BRK instruction.
- **RTI** returns from an interrupt service routine.
- This instruction restores the processor flags from the stack and restores the PC register.
- After the processor flags are restored, the **B** flag is not meaningful and *should be ignored*.

— Computer Architecture

Chapter 3: Processor Elements

3. Instruction categories

- **No operation**
- The **NOP** instruction (often referred to as no-op) does nothing except advance the PC register to the following instruction.
- NOP instructions are sometimes used as a **debugging tool** during program development.
- For example, one or more instructions can be effectively "commented out" by filling the memory addresses for those instructions with **\$EA** bytes.
- **\$EA** is the hexadecimal value of the 6502 **NOP** opcode.

— Computer Architecture

Chapter 3: Processor Elements

Contents

1. A simple processor
2. Addressing modes
3. Instruction categories
- 4. Interrupt processing**
5. Input/output operations

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/IRQ processing**
- The 6502 integrated circuit has two input signals that allow external components to notify the processor of a need for attention. The first is the **interrupt request input, IRQ**.
- IRQ is an **active low** (that's what the bar over the IRQ characters means) input that generates a processor interrupt when pulled **low**.
- The 6502 cannot respond instantly to a low signal level on the IRQ input. Before the 6502 can begin to process the interrupt, *it must first complete the instruction already in progress*.
- Next, it pushes the return address (the address of the next instruction that would have been executed after the instruction in progress) onto the stack, followed by the processor flags register. Since this interrupt was generated by the IRQ input, the **B** flag in the processor flags on the stack will be **0**.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/IRQ processing**
- Unlike the **JSR** instruction, the return address pushed in response to the IRQ input is the actual address of the next instruction to be executed, rather than the instruction address minus 1. The interrupt return address will not be incremented to generate the return address as occurs during RTS instruction execution.
- In the next stage of interrupt processing, the processor loads the **address** of the **IRQ handler routine** from memory addresses **\$FFFE-\$FFFF** into the PC register. The 6502 then begins executing the interrupt handler code at that address.
- When the interrupt handler is finished, it executes the **RTI** instruction. RTI pops the processor flags and the PC register values from the stack and resumes execution at the instruction following the instruction that was in progress when the IRQ input was driven low.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/IRQ processing**
- The IRQ input is a **maskable** interrupt, meaning it is possible to perform the equivalent of putting the telephone ringer on mute. When IRQ processing begins, the 6502 automatically **sets** the **I** flag, which masks (**disables**) the IRQ input until the I flag is cleared.
- The **I** flag will be cleared by the RTI instruction because the I flag could not have been set when the processor began responding to the IRQ. The **I** flag can also be cleared by the **CLI** instruction, which means it is possible to **enable** IRQ interrupts while processing an IRQ interrupt. An interrupt handled while processing another interrupt is referred to as a **nested interrupt**.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/IRQ processing**
- The IRQ input is **level-sensitive**, which means any time the IRQ input is **low** and the **I** flag is **cleared**, the processor will initiate the interrupt processing sequence. The consequence of this is that, at the completion of processing an interrupt, the 6502's interactions with the interrupt source must ensure the IRQ input is no longer low.
- If IRQ remains low when the RTI instruction is executed, the 6502 will immediately begin the interrupt handling process all over again.
- Interrupts initiated via the IRQ input handle most routine interactions between the 6502 and peripheral devices.
- For example, the **keyboard** is an interrupt source in most computers. Each keypress generates an IRQ interrupt. During keyboard interrupt processing, the 6502 reads the identification of the key from the keyboard interface and stores it into a queue for later processing by the currently active application.
- The IRQ handler code does not need to know anything about what the key press information will be used for; it just saves the data for later use.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/NMI processing**
- The second interrupt input to the 6502 is the **non-maskable interrupt, NMI**.
- As its name implies, the NMI input is not masked by the **I** flag. NMI is an **edge-sensitive** input that triggers on the **falling edge** of the signal.
- The processing of NMI interrupts is similar to the processing of IRQ interrupts, except the address of the interrupt handler routine is loaded from memory addresses **\$FFFA-\$FFFB** and the **I** flag has no effect on this type of interrupt.
- Because NMI is **non-maskable**, it can be triggered at any time, including when the 6502 is in the middle of handling an IRQ interrupt, or even while handling an earlier NMI interrupt.
- The NMI input is normally reserved for *very high priority conditions* that cannot be delayed or missed. One possible use of NMI interrupts is to trigger the incrementing of a **real-time clock** at regular intervals.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **/NMI processing**
- This NMI handler code increments a 32-bit clock counter located at addresses \$10-\$13 each time the interrupt occurs:

```
; Increment a 32-bit clock counter at each /NMI interrupt
NMI_HANDLER:
INC $10
BNE NMI_DONE
INC $11
BNE NMI_DONE
INC $12
BNE NMI_DONE
INC $13
NMI_DONE:
RTI
```

Figure 3.9: Example of a /NMI handler

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **BRK instruction processing**
- The **BRK** instruction triggers processing similar to an IRQ interrupt.
- Because BRK is an instruction, there is no need to wait for the completion of an instruction in progress before initiating interrupt processing.
- During BRK execution, the **return address** (the address of the BRK instruction plus 2) and the **processor flags** are pushed onto the stack, similar to the response to a low level on the IRQ input.
- Note that by **adding 2** to the BRK instruction address, the return address is not pointed to the byte after BRK, but to the second byte after it.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **BRK instruction processing**
- The BRK instruction is **non-maskable**: the state of the I flag does not affect the execution of the BRK instruction.
- The BRK handler address is the same as the IRQ handler, which is located at memory addresses **\$FFFE-\$FFFF**.
- Since the BRK instruction and IRQ share the same handler, the **B** flag must be consulted to identify the interrupt source during processing. The **B** flag in the processor flags pushed onto the stack (note that this is not the B flag in the processor flags (P) register) will be **set** in response to a **BRK** instruction and it will be **clear** during the processing of an **IRQ** interrupt.
- The BRK instruction finds little use in most 6502 applications. A traditional usage for this instruction is to **set breakpoints** when *debugging a program*. By temporarily replacing the opcode byte at the desired break location with a BRK instruction, it is possible for the debugging program (often called a **monitor** in smaller computer systems) to gain control and allow the user to display and modify register contents and memory contents and then resume execution.

— Computer Architecture

Chapter 3: Processor Elements

4. Interrupt processing

- **BRK instruction processing**
- This code example implements a minimal IRQ handler that differentiates between IRQ interrupts and BRK instructions. It uses memory address \$14 as a temporary storage location dedicated for use by this routine:

```
; Handle /IRQ interrupts and BRK instructions
IRQ_BRK_HANDLER:
; Save the A register
STA $14

; Retrieve the processor flags from the stack into A
PLA
PHA

; Check if the B bit is set in the flags on the stack
AND $10 ; $10 selects the B bit

; If the result is nonzero, B was set: Handle the BRK
BNE BRK_INSTR

; B was not set: Handle the /IRQ here
; ...

JMP IRQ_DONE
```

```
BRK_INSTR:
; Handle the BRK instruction here
; ...

IRQ_DONE:
; Restore the A register and return
LDA $14
RTI
```

Figure 3.10: Example of a BRK handler

— Computer Architecture

Chapter 3: Processor Elements

Contents

1. A simple processor
2. Addressing modes
3. Instruction categories
4. Interrupt processing
- 5. Input/output operations**

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Memory-mapped I/O and port-mapped I/O**
- **Memory-mapped I/O** and **port-mapped I/O** are the two main approaches employed in modern processors to access I/O devices.
- **Memory-mapped I/O** dedicates portions of the system address space to I/O devices. The processor accesses peripheral devices at predefined addresses using the same instructions and addressing modes it uses to read and write system memory.
- The **6502** uses **memory-mapped I/O** to communicate with its peripherals.
- Processors using **port-mapped I/O** implement a separate category of instructions for performing I/O operations. Port-mapped I/O devices have a dedicated address space independent of system memory. I/O devices are assigned port numbers as addresses.
- The **x86** architecture employs **port-mapped I/O**.
 - One drawback of **memory-mapped I/O** is the need to dedicate part of the system address space to I/O devices, which reduces the maximum amount of memory that can be installed in the computer system.
 - A drawback of **port-mapped I/O** is the requirement for the processor to implement additional instructions to perform the I/O operations.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Memory-mapped I/O and port-mapped I/O**
- Some implementations of **port-mapped I/O** provide **additional hardware signals** to indicate when an I/O device is being addressed as opposed to system memory.
- Using this signal as a selector (essentially another address bit), the same address lines can then be used for accessing memory and I/O devices.
- Alternatively, some higher-end processors implement a **completely separate bus** for performing **port-mapped I/O** operations. This architecture allows I/O and memory access operations to proceed simultaneously.
- In the simplest approach to I/O, the processor handles all steps in an I/O operation itself, using instructions to transfer data between memory and the I/O device.
- More complex processor architectures provide hardware features to accelerate repetitive I/O operations.
- We will now discuss three methods of performing I/O with varying degrees of processor involvement: **programmed I/O**, **interrupt-driven I/O**, and **direct memory access**.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Programmed I/O**
- **Programmed I/O** simply means *the processor performs every step of the I/O data transfer operation itself* using program instructions.
- Consider a **keyboard** that presents itself to the processor as two memory-mapped one-byte addresses in the processor's I/O address region.
- One of these bytes contains **status information**, specifically a bit indicating when a key has been pressed. The **second** byte contains the value of the key that was pressed.
- Each time a key is **pressed**, the key available status bit is **set**. When using programmed I/O, the processor must periodically read the keyboard **status register** to see whether a key has been pressed. If the status bit indicates a key has been pressed, the processor reads the keyboard data register, which turns off the key available status bit until the next keypress occurs.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Programmed I/O**
- If the keyboard data register can only hold one key at a time, this keyboard status checking operation must occur **frequently enough** that no key presses get lost, even when a very fast typist is at the keyboard. As a result, the processor must spend a significant amount of its time checking to see whether a key has been pressed. Most of these checks will turn out to be fruitless whenever fast typing is not taking place.
- It should be clear that programmed I/O is *not a very efficient method for general use*. It is similar in concept to checking your phone every few seconds to see whether someone is calling you.
- The use of programmed I/O makes sense in some situations. For example, the **one-time configuration** of a **peripheral device during system startup** is a reasonable use of this technique.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Interrupt-driven I/O**
- An I/O device can use **interrupts** to notify the processor when action is needed.
- In the case of the simple keyboard interface, instead of merely setting a bit in a status register, the peripheral could pull the 6502's IRQ line low to initiate an interrupt each time a key is pressed.
- This allows the processor to go about its business without checking for keypresses. The processor will only focus attention on the keyboard interface when there is work to be done, as indicated by the interrupt. Using interrupts to trigger I/O operations is analogous to adding a ringer to the phone that we had to check for incoming calls every few seconds when using programmed I/O.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Interrupt-driven I/O**
- The 6502 has a single **maskable** interrupt input signal (IRQ) available for I/O operations.
- Computer systems usually contain multiple sources of I/O interrupts. This makes the task of servicing interrupts a bit more complicated in the 6502 because the processor must first identify which peripheral initiated the interrupt before it can begin transferring data.
- The **interrupt service routine** must poll each interrupt-capable device to locate the interrupt source. In the case of the keyboard interface, this polling operation consists of reading the keyboard status register to determine whether the bit is set, indicating a keypress occurred.
- Once the processor has identified the device responsible for the interrupt, it branches to code that interacts with the device to complete the requested I/O task.
- In the case of the keyboard interface, this processing performs the steps of reading the keyboard data register and clearing the key available status bit, which deactivates the IRQ input signal.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Interrupt-driven I/O**
- Interrupts from external devices are **asynchronous** events, meaning they can occur at any time.
- Computer system design must include careful consideration of the possibility that interrupts may be generated at potentially unexpected times such as during system startup or while processing other interrupts. Interrupts from multiple devices may occur **simultaneously**, or **nearly simultaneously**, and in **random** order.
- Interrupt handling hardware circuitry and interrupt servicing code must ensure that all interrupts are detected and processed regardless of any timing peculiarities.
- **Interrupt-driven I/O** eliminates the processor's need to periodically check I/O devices to see whether action is needed. However, handling an interrupt may consume significant processor time if it involves transferring a large block of data, such as when reading from or writing to a disk drive.
- The next I/O method we will discuss removes the need for the processor to perform the work of transferring large blocks of data.

— Computer Architecture

Chapter 3: Processor Elements

5. Input/output processing

- **Direct memory access**
- **Direct Memory Access (DMA)** permits peripheral device I/O operations to access system memory independent of the processor.
- When using DMA to transfer a block of data, the processor sets up the operation by configuring a DMA controller with the **starting address** of the data block to be transferred, the **block length**, and the **destination address**.
- After initiating the DMA, the processor is free to continue other work. At the completion of the operation, the DMA controller generates an interrupt to inform the processor the transfer is complete.
- Within a computer system, a DMA controller may be implemented as a separate integrated circuit managed by the processor, or a processor architecture may contain one or more **integrated DMA controllers**.

— Computer Architecture

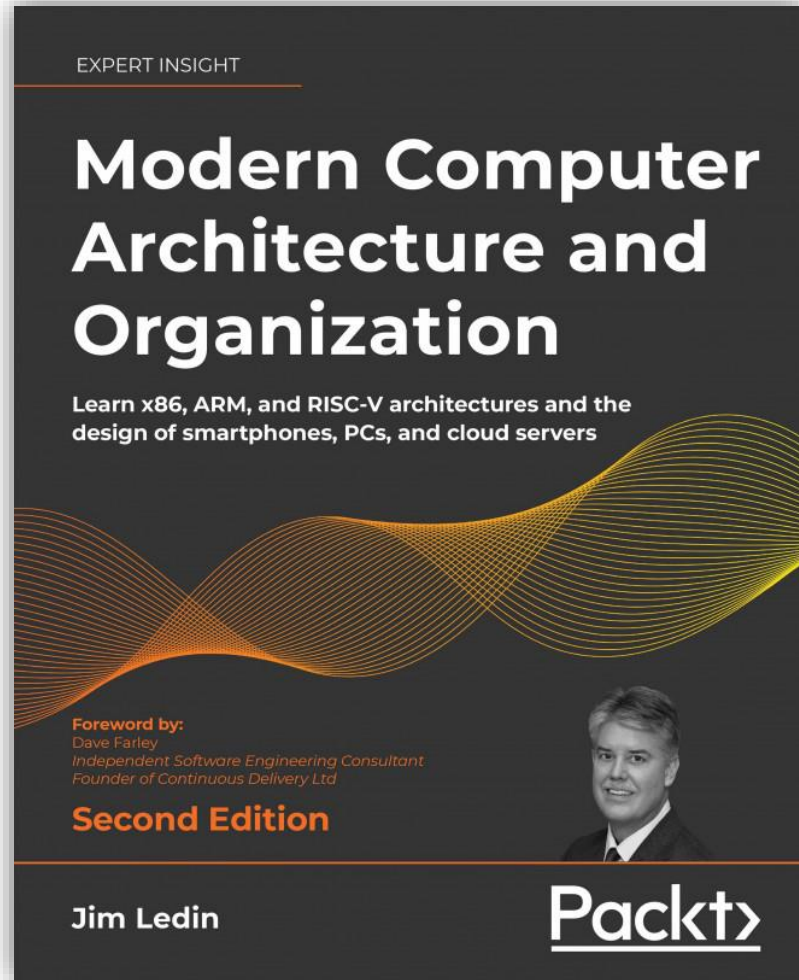
Chapter 3: Processor Elements

5. Input/output processing

- **Direct memory access**
- I/O devices that move substantial amounts of data, such as disk drives, sound cards, graphics cards, and network interfaces, generally rely on DMA to efficiently transfer data into and out of system memory. *DMA is also useful for transferring blocks of data within system memory.*
- The **6502 architecture** does **not** support DMA operations, but the original IBM PC included a DMA controller, and almost every 32-bit and 64-bit processor architecture provides extensive support for DMA operations.
- **DMA** is one of many techniques that improve computer system performance by accelerating repetitive operations.

Computer Architecture

End of Chapter 3 – Processor Elements



VIVES University of Applied Sciences
Bachelor in electronics-ICT

